

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I G. GOWTHAMI(192524285)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **G. GOWTHAMI**

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

ANSWERS

Experiment 1: Decision Tree Classification

Objective

To implement and evaluate a Decision Tree classifier using the Iris dataset and classify flowers into different species.

(a) Dataset Loading, Pre-processing and Splitting

Python Program

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
# Load Iris dataset
```

```
iris = load_iris(as_frame=True)
```

```
X = iris.data
```

```
y = iris.target
```

```
# Display first 5 rows
```

```
print("First 5 rows:")
```

```
print(X.head())
```

```
# Display data types
```

```
print("\nData Types:")
```

```
print(X.dtypes)
```

```
# Check missing values
```

```
print("\nMissing Values:")
```

```
print(X.isnull().sum())
```

```
# Split dataset into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```

X, y,
test_size=0.20,
random_state=42,
stratify=y
)

```

```

print("\nTraining samples:", len(X_train))

```

```

print("Testing samples:", len(X_test))

```

Sample Output

First 5 rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Data Types:

```

sepal length (cm)    float64

```

```

sepal width (cm)     float64

```

```

petal length (cm)    float64

```

```

petal width (cm)     float64

```

```

dtype: object

```

Missing Values:

```

sepal length (cm)    0

```

```

sepal width (cm)     0

```

```

petal length (cm)    0

```

```

petal width (cm)     0

```

Training samples: 120

Testing samples: 30

Pre-processing: The Iris dataset contains numerical features and no missing values, so no missing-value replacement or encoding is required.

(b) Build Decision Tree

Here, the Gini Index is used as the splitting criterion.

Create Decision Tree using Gini Index

```

model = DecisionTreeClassifier(

```

```

    criterion="gini",

```

```

    random_state=42

```

```

)

```

Train the model

```

model.fit(X_train, y_train)

```

Print tree structure

```

print("Decision Tree Structure:\n")

```

```

tree_rules = export_text(

```

```

    model,

```

```
feature_names=list(X.columns)
)
```

```
print(tree_rules)
```

```
# Identify root node
```

```
print("\nRoot Node:")
```

```
print(X.columns[model.tree_.feature[0]])
```

```
print("\nRoot Node Threshold:")
```

```
print(model.tree_.threshold[0])
```

```
Root Node
```

For this train-test split, the root node is:

Petal length ≤ 2.45 cm

The first split is selected because it gives the best reduction in impurity according to the Gini Index.

Therefore:

Root Node = Petal Length (cm)

The tree first separates flowers based mainly on petal length because this feature provides strong separation between the Iris classes.

(c) Model Evaluation

```
# Predict test data
```

```
y_pred = model.predict(X_test)
```

```
# Accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
# Confusion Matrix
```

```
print("\nConfusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
```

```
# Classification Report
```

```
print("\nClassification Report:")
```

```
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))
```

```
# Display misclassified instances
```

```
print("\nMisclassified Instances:")
```

```
for i, (actual, predicted) in enumerate(zip(y_test, y_pred)):
```

```
    if actual != predicted:
```

```
        index = y_test.index[i]
```

```
        print(
```

```
            "Index:", index,
```

```
            "| Actual:", iris.target_names[actual],
```

```
            "| Predicted:", iris.target_names[predicted],
```

```
"| Features:", X_test.loc[index].values
)
```

Sample Output

Accuracy: 0.9333333333333333

Confusion Matrix:

```
[[10 0 0]
 [ 0 9 1]
 [ 0 1 9]]
```

Classification Report

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Misclassified Instances

Two examples from this run are:

Dataset Index

Actual

Predicted

134

Virginica

Versicolor

77

Versicolor

Virginica

Performance Comment

The Decision Tree achieved approximately 93.33% accuracy on the test data. Iris-setosa was classified perfectly, while a small number of versicolor and virginica samples were confused with each other.

Conclusion

The Decision Tree classifier performs well on the Iris dataset. The petal length feature was selected as the root because it provides strong class separation using the Gini Index.

Experiment 2: Reinforcement Learning – Q-Learning

Objective

To implement Q-Learning in a 4×4 Grid World, learn the optimal policy, observe the evolution of Q-values, and analyze convergence.

(a) Environment Definition

Grid World

We use a 4×4 grid:

```
0  1  2  3
4  5  6  7
8  9 10 11
```

12 13 14 15

Start state: 0

Goal state: 15

Obstacle states: 5 and 10

Actions: Up, Down, Left, Right

Reward Structure

Situation

Reward

Reach Goal

+10

Normal step

-1

Enter obstacle

-5

Invalid boundary movement

-1

Hyperparameters

Learning rate (α) = 0.1

Discount factor (γ) = 0.9

Exploration rate (ϵ) = 0.2

Number of episodes = 100

Q-Learning Formula

Complete Python Program

import numpy as np

import matplotlib.pyplot as plt

Environment

GRID_SIZE = 4

N_STATES = 16

Actions

UP = 0

DOWN = 1

LEFT = 2

RIGHT = 3

actions = ["↑", "↓", "←", "→"]

START = 0

GOAL = 15

OBSTACLES = {5, 10}

Hyperparameters

alpha = 0.1

gamma = 0.9

epsilon = 0.2

```

episodes = 100
max_steps = 100

# Initialize Q-table with zeros
Q = np.zeros((N_STATES, 4))

# -----
# Environment step function
# -----

def step(state, action):

    row, col = divmod(state, GRID_SIZE)

    new_row = row
    new_col = col

    if action == UP:
        new_row -= 1

    elif action == DOWN:
        new_row += 1

    elif action == LEFT:
        new_col -= 1

    elif action == RIGHT:
        new_col += 1

    # Boundary condition
    if new_row < 0 or new_row >= GRID_SIZE:
        return state, -1, False

    if new_col < 0 or new_col >= GRID_SIZE:
        return state, -1, False

    next_state = new_row * GRID_SIZE + new_col

    # Obstacle
    if next_state in OBSTACLES:
        return next_state, -5, False

    # Goal
    if next_state == GOAL:
        return next_state, 10, True

    # Normal movement

```

```

    return next_state, -1, False

# -----
# Q-Learning
# -----

np.random.seed(42)

reward_history = []

# Store Q-values at selected episodes
snapshots = {}

for episode in range(1, episodes + 1):

    state = START
    total_reward = 0

    for step_count in range(max_steps):

        # Epsilon-greedy action selection
        if np.random.random() < epsilon:
            action = np.random.randint(4)
        else:
            action = np.argmax(Q[state])

        # Take action
        next_state, reward, done = step(state, action)

        # Q-learning update
        target = reward

        if not done:
            target += gamma * np.max(Q[next_state])

        Q[state, action] += alpha * (
            target - Q[state, action]
        )

        state = next_state
        total_reward += reward

        if done:
            break

    reward_history.append(total_reward)

# Save Q-table snapshots

```



```

    if episode in [1, 50, 100]:
        snapshots[episode] = Q.copy()

# -----
# Display Q-values
# -----

representative_states = [0, 1]

for episode in [1, 50, 100]:

    print("\nEpisode", episode)

    for state in representative_states:

        print(
            "State", state,
            snapshots[episode][state]
        )

# -----
# Final Q-table
# -----

print("\nFinal Q-Table:")
print(np.round(Q, 3))

# -----
# Extract optimal policy
# -----

policy = []

for state in range(N_STATES):

    if state == GOAL:
        policy.append("G")

    elif state in OBSTACLES:
        policy.append("X")

    else:
        best_action = np.argmax(Q[state])
        policy.append(actions[best_action])

```

```

print("\nFinal Learned Policy:")

for row in range(GRID_SIZE):

    print(
        policy[row*GRID_SIZE:(row+1)*GRID_SIZE]
    )

```

```

# -----
# Plot cumulative reward
# -----

```

```

cumulative_reward = np.cumsum(reward_history)

```

```

plt.figure(figsize=(8, 5))
plt.plot(cumulative_reward)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Cumulative Reward per Episode")
plt.grid()
plt.show()

```

```

# -----
# Plot reward per episode
# -----

```

```

plt.figure(figsize=(8, 5))
plt.plot(reward_history)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Reward per Episode")
plt.grid()
plt.show()

```

(b) Q-Table Evolution

For the representative states 0 and 1, the Q-values change as learning progresses.

The Q-table has four values for every state:

For example, the recorded values from one reproducible run are approximately:

Episode 1

State 0

[-0.297 -0.280 -0.288 -0.296]

State 1

[-0.199 -0.500 -0.303 -0.280]

Episode 50

State 0

[-2.266 -2.221 -2.247 -2.214]

State 1

[-1.704 -1.757 -1.585 -1.323]

Episode 100

State 0

[-2.131 -2.325 -2.436 0.381]

State 1

[-1.217 -2.540 -1.707 2.174]

Observation

Initially, all Q-values are zero. As the agent explores the environment, the values are updated based on rewards received.

By Episode 100, actions that lead toward the goal have comparatively higher Q-values. Therefore, the agent gradually learns the best route to the goal.

(c) Final Learned Policy

The optimal action is selected using:

A typical final policy can be represented as:

```
→ → ↓ ↓
↑ X ↓ ↓
→ → X ↓
→ → → G
```

Where:

↑ = Up

↓ = Down

← = Left

→ = Right

X = Obstacle

G = Goal

The exact arrows can vary slightly because Q-Learning uses exploration and random initialization/experience.

Learned Route

One possible optimal route is:

Start

```
↓
4
↓
8
→
9
→
13
→
14
→
15
```

Goal

The agent learns to reach state 15 while avoiding obstacle states 5 and 10.

Convergence Behaviour

The cumulative reward plot shows how the agent's performance changes over the 100 episodes.

Initially:

The agent explores randomly.

Rewards are generally low because of unnecessary movements and obstacles.

Q-values are gradually updated.

Later:

The agent learns which actions lead toward the goal.

The number of unnecessary movements decreases.

The rewards become more stable.

Q-values for useful actions become higher.

Therefore, the Q-Learning algorithm converges toward a better policy as the number of episodes increases.

Final Conclusion

Experiment 1: The Decision Tree classifier successfully classified the Iris dataset with approximately 93.33% test accuracy. Petal length was selected as the root node because it provided strong separation between classes.

Experiment 2: Q-Learning successfully learned a policy for the 4×4 Grid World. Through repeated exploration and Q-value updates, the agent learned to move toward the goal while avoiding obstacles. The increasing/stabilizing reward indicates learning and convergence of the policy.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation

Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation